ADL Final Report
team 72

R11922A12 李冠勳

R11922200 許郁翎

R10741071 簡睿閎

R10741056 譚庚豪

Abstract

The goal of this task is to predict the courses that would be purchased by users and subgroups that users belong to by observing the consuming records and subgroup labels in the past. The dataset includes the features of users and courses, as well as the subgroups which users belong to. The tasks are divided into two categories, which are seen users and unseen users. We have four members in our group. As we solve the tasks by using different methods, in the following discussion we will introduce our proposed methods individually.

Introduction

This grand challenging problem can be seen as an application of recommendation. In general, matrix factorization would be used to solve this kind of problem.

Yu-Ling Method : the features of users and courses would be turned into sentence embeddings using two encoders under the architecture of sentenceBERT. Then, cross attention will be done (here multi-head attention is used) on user embeddings and all course embeddings. Finally, the model will output the values of similarity between the user and all of the courses by doing weighted sum on the entries of similarity matrix obtained from cross attention. The higher value of the similarity score represents the more likely for the user to buy the course. To enhance the stability of the result, the ensemble method is introduced to generate the final prediction.

Jui-Hung & Geng-Hao Method : We use the same as matrix factorization but a lit bit different from Yu-ling,In data pre-processing, we divide user data and course data into three parts: continuous, discrete and text, and convert them into embedding as model input. For model selection, we use FiBiNet and its improved version FiBiNet++. Its architecture is mainly divided into two parts: SeNet and Bilinear , SeNet can give each field a weight to filter out important features and eliminate useless features. Similar to the operation of the attention mechanism, Bilinear is to perform feature crossover between fields, so that the model can learn the interaction information between users and courses. Due to limited hardware equipment, we choose to share feature cross parameters and use inner product instead of Hadamard product to reduce model parameters. Finally, on the base of FiBiNet we add layer norm after embedding layer, and add skip-connection and remove the Bilinear layer after Senet becomes an improved version of FiBiNet++. During training, we use a ratio of positive and negative samples of 1:1, and the output of the ranking model takes the top 50 as the final prediction result.

Kuan-Hsun Method: The course prediction can be split into two parts. 1. relative course filter 2. relative course ranking. For the first part, we get the corresponding course of each user. Then, we predict the ranking of the corresponding classes. How to select the feature? For user features, the interest and gender column is one hot encoded. And, the description in the course feature is transformed into embedding by SentenceBert and reduced to 100 dimensions by PCA. After we have user embedding and course embedding, we cross join them and get a user x course's matrix. Finally we can predict the course ranking by LightGBM

Related work & Approach

In Yu-Ling's approach, the intact architecture can be separated into three submodules, which are two sentence encoders which turn features of uses and courses into embeddings and a model which estimates the similarity between users and courses. The training process can be divided into three parts: finetune sentence encoder, cross attention and ensemble.

<u>finetune sentenceBERT</u>
The concept of sentenceBERT is from the paper, Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. given two sentences and their similarity score, the sentence encoder would be trained to generate the sentence embeddings that the similarity score calculated to be close to the target label. Here, we introduce how we design labels to finetune user encoders and course encoders in the following:

1. The similarity score between two users is proportional to the number of common courses they bought.
2. The similarity score between two courses is proportional to the number of customers who bought both of them.

<u>cross attention</u>
After turning the features of users and courses into embedding vectors, we will do cross attention on these embeddings. the user embeddings will be the queries, and on the other hand, the course embeddings will be the keys. The similarity matrix generated from cross attention will finally pass through a linear layer, and the model will output a similarity score vector representing the extent of how likely the user would buy the courses.

<u>label design</u>
In the experiment, we design the ground truth label of the similarity between the user and the course by setting it as $decay\ ratio^n$, where n is the ranking of the course.

In the subgroup prediction tasks that Geng-Hao and Jui-Hung were in charge of, it can be divided into three parts：

1. Data Preprocessing

- Course & User Metadata：Mainly divide into continuous, discrete and text for processing
    - Continuous: We tried to use MinMaxScaler, StandardScaler and binning discretization, and finally found that the binning discretization model performed better.
    - Discrete: Use LabelEncoder for classification, each category corresponds to an index and each index represents a unique Embedding, Like an Embedding dict.
    - Text: Use the Pretrained Sentence Model provided by HuggingFace to map text features to an Embedding.

  After multiple feature cross-feeding models, the features that have more influence on the performance of the model, such as Course Price (continuous), Description (text), Subgroup (discrete), Group (discrete), Gender (discrete), and Interest (discrete), are selected.
- Create Negative Sample for Training : In order to avoid the problem of overfitting, the strategy of positive samples and negative samples is adopted in the part of the training data set for preprocessing, for each user that appears in the "subgroup" column is regarded as positive sample and marked as 1,on the contrary, if it is not in the subgroup column, it is regarded as a negative sample and marked as 0, in the process of selecting negative samples, we imitated the practice of word2vec paper, and took the popularity of subgroups to the power of 0.75 and converted them into probability forms as the probability of being selected as negative samples. We hope that relatively unpopular subgroups can also be selected to the opportunity, Finally, we counted more than 470,000 data for training.
- Validation & Test data : Because it is a multi-label problem, we hope that each user can correspond to different subgroups one-to-one, so a 92-dimensional (total number of subgroups, ‘0’ for No subgroup) vector is generated for each user, and that appear in the "subgroup" column are marked as 1, and those that do not appear are all marked as 0.

2. Model Architecture

FiBiNet :

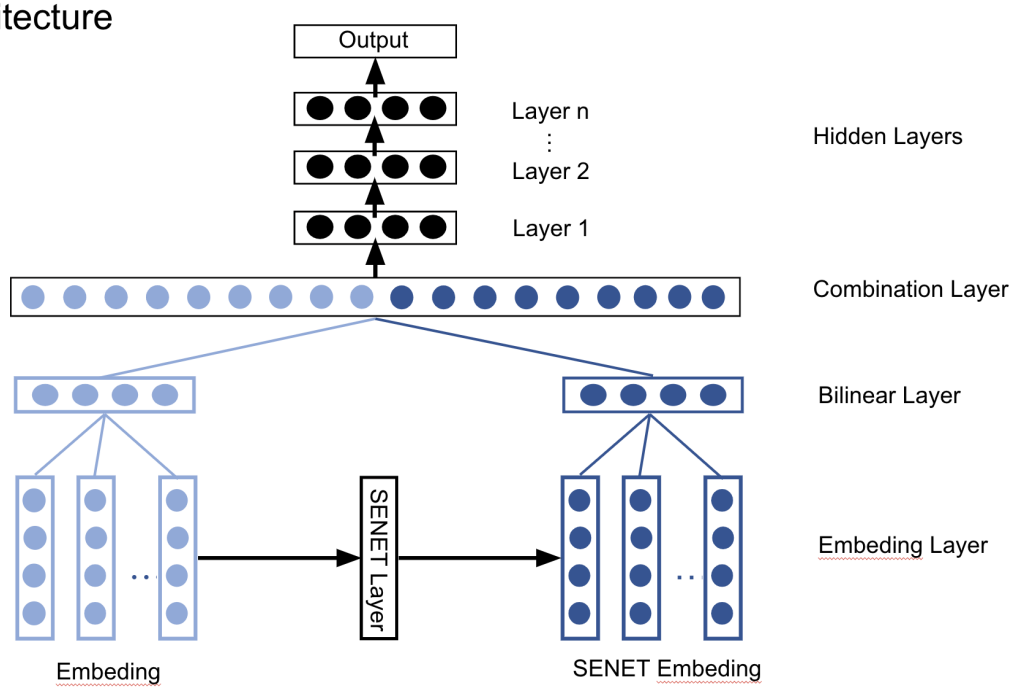
- Embedding layer：We use Pytorch api nn.Embedding to create a randomized embedding to be used as model input, and passes through a linear SeNet layer. This linear layer generates a feature weighted vector and multiplies the input vector to weight important features and reduce the importance of other features. The process mainly includes three steps
    - Squeeze：$z_i = F_{sq}(e_i) = \frac{1}{k}\sum_{t=1}^{k} e_i^{(t)}$ perform mean pooling on each dimension of the original embedding, and finally get output vector(z_i)
    - Excitation：$A = F_{ex}(Z) = \sigma_2(W_2\sigma_1(W_1 Z))$ use the vector obtained by squeeze to learn feature weights according to the data compression of the feature, and finally get the weight vector (A) to be multiplied with the embedding, where σ1, σ2 are activate functions, W1, W2 are parameters
    - Reweight：$V = F_{ReWeight}(A, E) = [a_1 \cdot e_1, \ldots, a_f \cdot e_f] = [v_1, \ldots, v_f]$ multiply the weights obtained by the original embedding and excitation, and finally get a new embedding
- Bilinear layer：$p_{ij} = v_i W \odot v_j$ Based on traditional inner product and Hadamard product, but introduce a parameter matrix (W) between the two to achieve the purpose of cross-learning, multiply Vi and W matrix to get a vector with the same dimension as Vj, and then use this vector and Vj to perform Hadamard product
- Combination layer：From the above two processes, a set of vectors obtained by passing the original embedding and SeNet embedding through the interaction layer respectively, this step concatenates the set of vectors together, and finally obtains a combination output
- DNN(Hidden) layer：Following the combination layer in the previous step, we put the concatenate vectors into a DNN model with three linear layers, the hidden_size is [128, 128, 1] and finally output a value between 0 and 1 through sigmoid for the degree of fit between users and subgroups.

FiBiNet++ :

- Embedding layer : Add a LayerNorm after every embedding layer.
- Reweight : Add a skip-connection, add the original embedding and new embedding and have a LayerNorm after that.
- Bilinear layer : Use inner product instead of Hadamard product and remove the bilinear layer after SeNet.

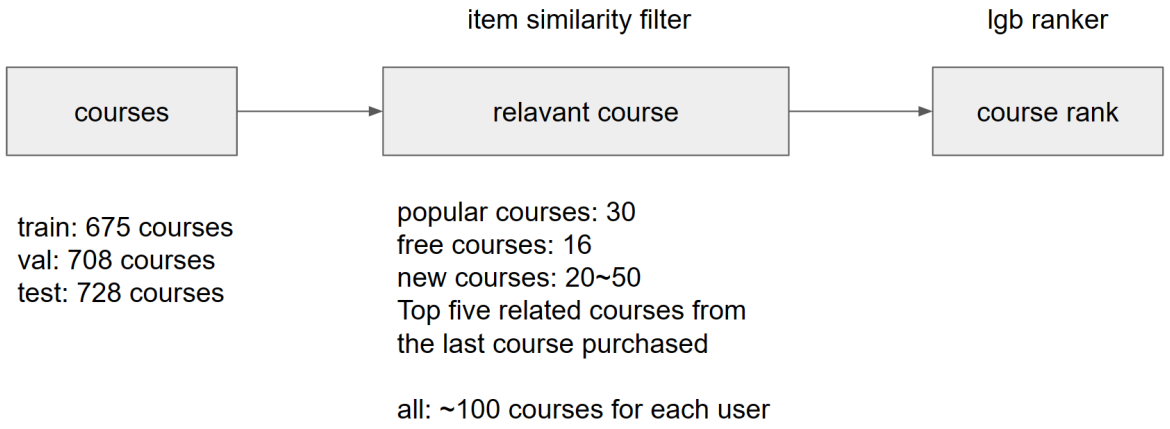
(below is architecture of FiBiNet)

Model Architecture

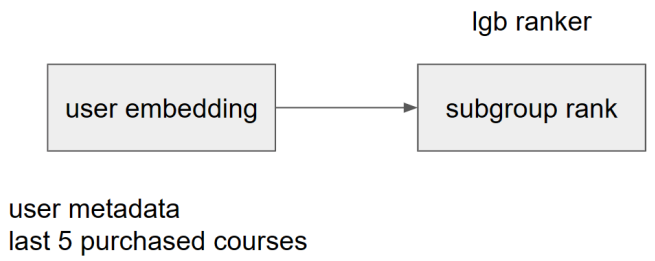


LightGBM :

Course Prediction Architecture



SubGroup Prediction Architecture



Experiments

- FiBiNet & FiBiNet++ Parameters Settings:
  - Train batch size: 1024
  - Learning rate: 1e-3
  - num_epochs: 5
  - weight_decay: 1e-2
  - embedding size: 16
  - optimizer: AdamW
  - lr_scheduler: CosineAnnealingLR(eta min: Learning rate * 1e-2)
  - top_k: 50
  - SeNet reduction_ratio: 3
  - DNN hidden units: [128, 128, 1]
  - DNN dropout: 0.2
  - Loss Function: BinaryCrossEntropy(reduction='sum')
  - Test batch size: 256
- Cross Attention Parameters Settings:
  - Batch size: 10
  - Optimizer: AdamW
  - Criterion: MSELoss.

LightGBM parameters are set as follow

```
{
    'objective': 'lambdarank',
    'task': 'train',
    'boosting_type': 'gbdt',
    'num_class': 1,
    'metric': {'map'},
    'num_leaves': 255,
    'learning_rate': 0.035,
    'max_bin': 2047,
    'subsample_for_bin': 200000,
    'reg_alpha': 0.0,
    'reg_lambda': 0.0,
    'max_depth': -1,
    'min_child_samples': 200,
```

```
    'min_child_weight': 0.001,
    'feature_fraction': 0.9,
    'bagging_fraction': 0.8,
    'bagging_freq': 2,
    'force_col_wise':'true',
    'verbose': 0,
    'eval_at': 50,
    'feature_fraction': 0.8,
    'num_threads': 32,
    "first_metric_only": True,
    'early_stopping_round': 200
}
```

The experiment results are as follows.
course：

task	method	validation score	public score	private score
course seen	lightGBM	0.08452	0.05329	0.05556
course seen	cross attention	0.0604	0.03534	0.03642
course unseen	lightGBM	0.124893	0.10189	0.10179
course unseen	cross attention	—	0.01876	0.01985

subgroup：

task	method	validation score	public score	private score
subgroup seen	lightGBM	0.2873	0.28964	0.28926
subgroup unseen	lightGBM	0.3552	0.32246	0.32995

Discussion

In the Cross Attention method proposed by Yu-Ling, the finetune stage didn't have a good effect on the result. We guess that the reason may lie in the inappropriate label design, due to the fact that all the pairs of user feature embeddings and course feature embeddings of each single user to a specific course share the same label. In the final submission model, I used the original version of sentenceBERT on the hugging face. Truly speaking, this design is not the best solution, for the number of

courses should be very large and dynamically changing merchandise in the actual marketing situation.

In our FiBiNet & FiBiNet++ method, we find that because of extending our valid and test data make the data too large so it will spend a lot of time training we just have two epoch valid result and didn't have a good effect so we didn't put the experiment result into the table above.

Conclusion

Under the framework of the modern recommender systems, we found that, during the training process, usually the inputs(after labeling) fed into the model are sparse matrices, and the boosting model does not need to be preprocessed for the features we have selected. Instead, it adds weak classifiers to the strong classifiers, and re-adjust the data weights, that is, assign higher weights to misclassified data, and reduce the weight of correctly classified data to achieve the purpose of paying more attention to misclassified data. Based on the above, we think that is the reason why boosting usually has better performance than neural networks,so eventually we choose gradient boosting models as our training model.

Work distribution

- Courses Tasks：
    - Seen Courses: 李冠勳、許郁翎
    - Unseen Courses: 李冠勳
- Subgroup Tasks：
    - Seen Topic: 李冠勳、簡睿閎、譚庚豪
    - Unseen Topic: 李冠勳、簡睿閎
- ppt 製作：李冠勳、許郁翎、譚庚豪
- 影片錄製和報告：李冠勳、許郁翎、簡睿閎
- Report撰寫：李冠勳、許郁翎、譚庚豪、簡睿閎